

Deployment Verification Procedure Report

Introduction

This playbook documents the deployment verification procedures performed by OpenZeppelin to review Everdawn's contracts (e.g., USDT0 and XAUt0) deployed on-chain. The primary objective of this activity is to ensure that the transition from an audited codebase to a production environment is executed without errors or misconfigurations.

Proper deployment verification helps identify issues such as incorrect initialization parameters and proxy configurations, mismatched bytecode, improperly assigned privileges, and incompatible EVM settings before they can be exploited or cause operational failures. By systematically reviewing each aspect of the deployment process, we provide assurance that the deployed contracts match their intended specifications and are configured securely.

To date, OpenZeppelin has completed 7 deployment review projects for USDT0. These engagements resulted in the successful verification of 16 token contracts across the designated networks.

Deployment Review Methodology

The deployment review process consists of a comprehensive series of steps designed to confirm that all contracts and their initializations are correct on-chain. This verification is performed through a combination of on-chain data analysis and a thorough code review of the migration scripts provided by the development team.

Initialization Parameter Verification

A granular analysis is conducted on all initialization parameters passed to the smart contracts. This level of detail is required to ensure that the protocol state is consistent with the intended design.

- **Parameter Order Verification:** All arguments are checked to ensure they are passed in the correct sequence to the constructor or initializer function. This prevents issues where values of the same type might be swapped accidentally.

- **Data Encoding Validation:** Parameters are verified to ensure they are correctly encoded according to the ABI requirements. This includes a review of the formatting for complex types and byte arrays.
- **Value Consistency:** Values used in the deployment are compared against the expected technical specifications. This ensures that constants, decimals, and protocol specific variables match the intended design.
- **Address Verification:** Every address passed as a parameter is confirmed to exist on the specific target chain and verified to belong to the intended entities.
- **Privilege Assignment Accuracy:** A thorough check of the access control state is conducted. This ensures that ownership and specialized roles are granted only to trusted addresses and that no unauthorized accounts hold administrative power.



Where applicable, scheduled transactions in the Safe wallet related to the setup and configuration of these contracts are also reviewed upon request.

Bytecode and Structural Verification

Technical checks are performed to ensure the deployed code matches the audited source code exactly:

- **Bytecode Comparison:** The on-chain bytecode is verified against the expected bytecode derived from the audited source code. Discrepancies related to compiler versions or metadata are investigated thoroughly. This process is typically performed for the following six contracts: the token proxy, token ProxyAdmin, token implementation, OFT proxy, OFT ProxyAdmin, and OFT implementation.
- **Proxy and Admin Links:** Proxies are confirmed to point to the correct implementation addresses and proxies are configured to link to the correct ProxyAdmin.
- **OFT and LayerZero Configuration:** For Omnichain Fungible Tokens, the linkage to the correct token addresses is verified along with the accuracy of the LayerZero endpoint configuration for each specific chain.

Migration Script and EVM Compatibility

The verification process also extends to the infrastructure used to facilitate the deployment:

- **Script Logic Analysis:** A code review of the deployment scripts is performed to identify uninitialized variables or flaws in the execution flow. This prevents scenarios where a script might fail partially and leave the protocol in an inconsistent state.
- **EVM Opcode Compatibility:** Compatibility with the EVM (Ethereum Virtual Machine) version of each target chain is verified. This is especially important for chains that do not yet support certain opcodes to prevent execution failures.
- **Storage Collision Checks:** For any upgradeable components, reviews are conducted to ensure that the new implementation does not introduce storage layout conflicts with previous versions.

Deliverables Provided

A comprehensive technical report is delivered as the primary output of the verification process. This document starts with a concise Executive Summary that highlights the overall status of the deployment and all issues identified. Detailed results of the bytecode comparison are included to provide evidence that the on-chain code matches the audited source material exactly.

Chain specific findings are meticulously cataloged for both EVM and non-EVM environments to address any network peculiarities. Configuration consistency across all supported networks is also analyzed and presented within the report. This ensures that the protocol behaves uniformly regardless of the underlying blockchain infrastructure.

Recommended corrective actions are provided for any issues identified during the review phase. Furthermore, notes on long-term operational risks and potential improvements are included to assist the development team, helping in maintaining the security posture of the contracts well after the initial deployment.

Conclusion & Value for USDT0

This methodology provides high assurance that every on-chain deployment accurately reflects the audited source code. The risk of deploying unvetted or corrupted code is effectively eliminated by verifying the bytecode and initialization logic. This alignment between the audit phase and the production environment serves as a fundamental security pillar for the protocol.

Safe multi-chain expansion is facilitated by preventing configuration drift and maintaining uniformity across diverse network environments. Operational resilience is further strengthened by addressing specific chain requirements and compatibility issues before they affect the deployed contracts. This standardized approach ensures that the ecosystem remains robust and reliable regardless of infrastructure nuances during the scaling process.